



JAIDEV EDUCATION SOCIETY'S
J D COLLEGE OF ENGINEERING AND MANAGEMENT
KATOL ROAD, NAGPUR

Website: www.jdcoem.ac.in E-mail: info@jdcoem.ac.in

(An Autonomous Institute, with NAAC "A" Grade)

Affiliated to DBATU, RTMNU & MSBTE Mumbai

Department of Computer Science & Engineering

"A Place to Learn, A Chance to Grow"

Session: 2025-26



VISION

To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.

MISSION

1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities.
2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program.
3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

Practical No. 10

Aim: Build KNN Classification model for a given dataset.

Software Required: Python IDE, Anaconda, VS Code, Jupyter Nootbook, Google Colab.

Theory:

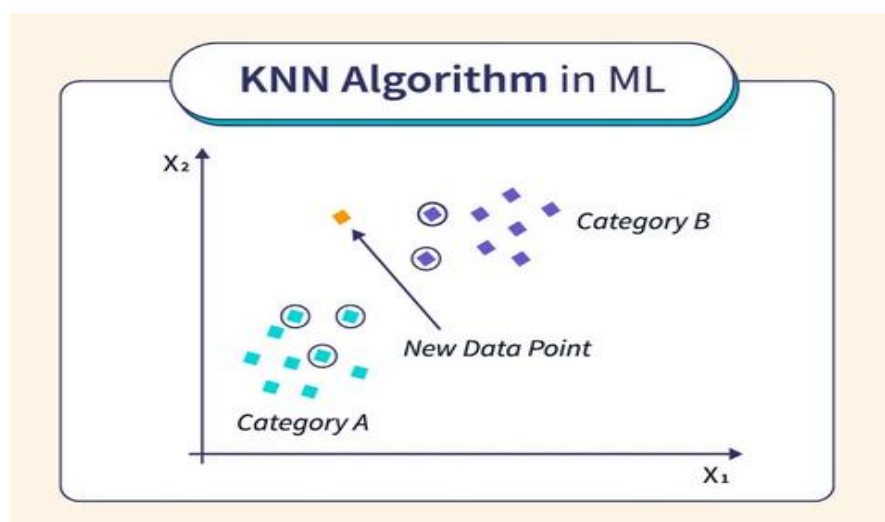
K-Nearest Neighbors (KNN) Algorithm

K-nearest neighbors (KNN) algorithm is a type of supervised ML algorithm which can be used for both classification as well as regression predictive problems. However, it is mainly used for classification predictive problems in industry. The main idea behind KNN is to find the k-nearest data points to a given test data point and use these nearest neighbors to make a prediction. The value of k is a hyperparameter that needs to be tuned, and it represents the number of neighbors to consider.

For classification problems, the KNN algorithm assigns the test data point to the class that appears most frequently among the k-nearest neighbors. In other words, the class with the highest number of neighbors is the predicted class.

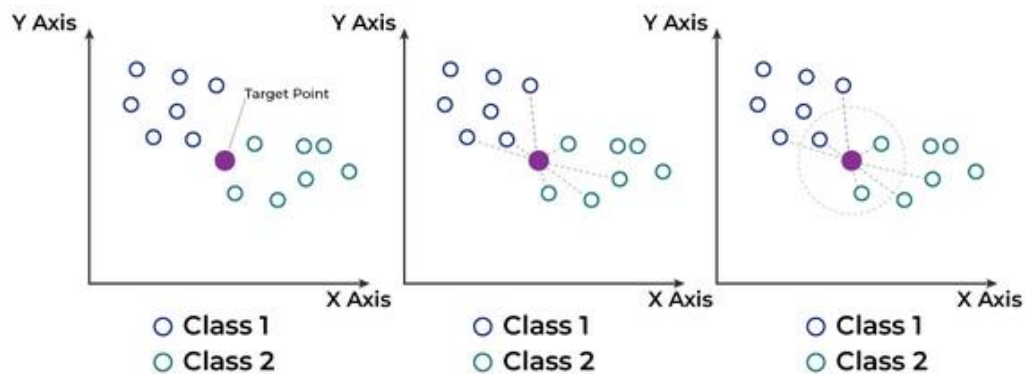
For regression problems, the KNN algorithm assigns the test data point the average of the k-nearest neighbors' values.

The distance metric used to measure the similarity between two data points is an essential factor that affects the KNN algorithm's performance. The most commonly used distance metrics are Euclidean distance, Manhattan distance, and Minkowski distance.



Working of KNN algorithm

The K-Nearest Neighbors (KNN) algorithm operates on the principle of similarity where it predicts the label or value of a new data point by considering the labels or values of its K nearest neighbors in the training dataset.



Step 1: Selecting the optimal value of K

- K represents the number of nearest neighbors that needs to be considered while making prediction.

Step 2: Calculating distance

- To measure the similarity between target and training data points Euclidean distance is widely used. Distance is calculated between data points in the dataset and target point.

Step 3: Finding Nearest Neighbors

- The k data points with the smallest distances to the target point are nearest neighbors.

Step 4: Voting for Classification or Taking Average for Regression

- When you want to classify a data point into a category like spam or not spam, the KNN algorithm looks at the K closest points in the dataset. These closest points are called neighbors. The algorithm then looks at which category the neighbors belong to and picks the one that appears the most. This is called majority voting.
- In regression, the algorithm still looks for the K closest points. But instead of voting for a class in classification, it takes the average of the values of those K neighbors. This average is the predicted value for the new point for the algorithm.

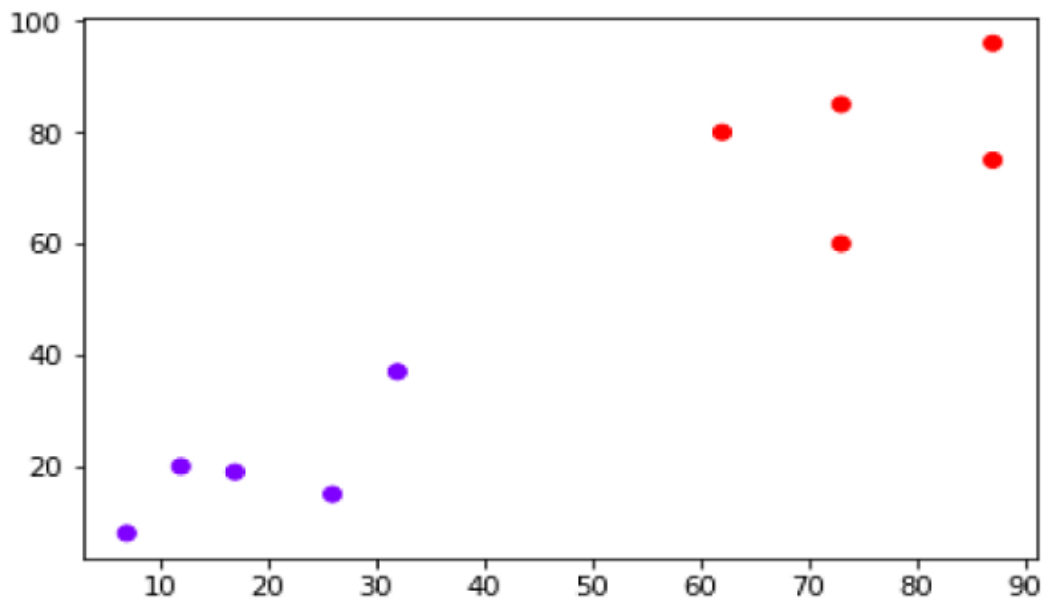
It shows how a test point is classified based on its nearest neighbors. As the test point moves the algorithm identifies the closest 'k' data points i.e. 5 in this case and assigns test point the majority class label that is grey label class here.

Example

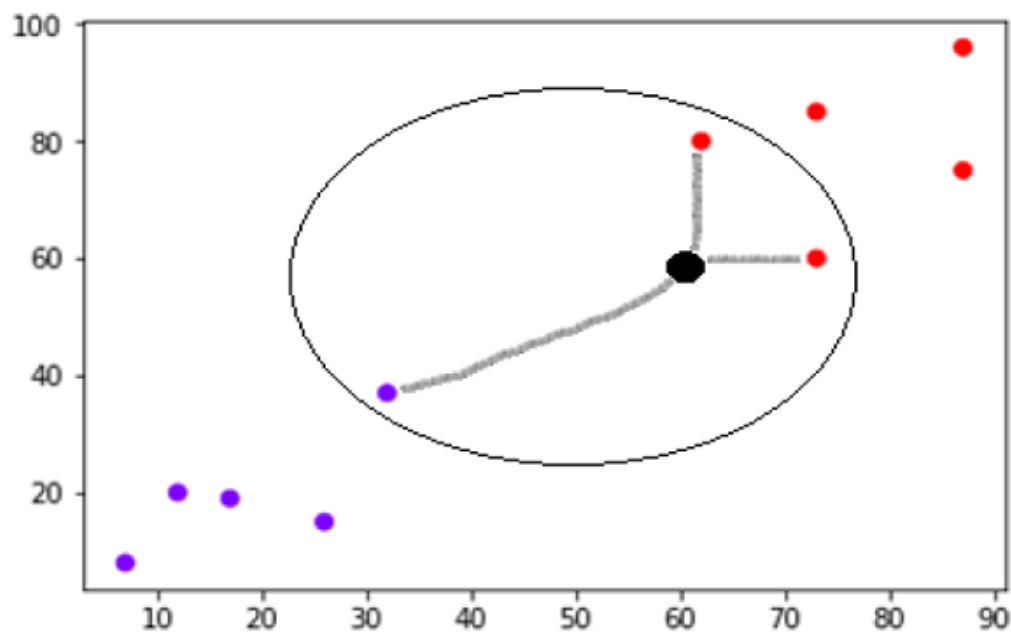
The following is an example to understand the concept of K and working of KNN algorithm

—

Suppose we have a dataset which can be plotted as follows —



Now, we need to classify new data point with black dot (at point 60,60) into blue or red class. We are assuming $K = 3$ i.e. it would find three nearest data points. It is shown in the next diagram –



We can see in the above diagram the three nearest neighbors of the data point with black dot. Among those three, two of them lies in Red class hence the black dot will also be assigned in red class.

Applications of KNN

- **Recommendation Systems:** Suggests items like movies or products by finding users with similar preferences.

- **Spam Detection:** Identifies spam emails by comparing new emails to known spam and non-spam examples.
- **Customer Segmentation:** Groups customers by comparing their shopping behavior to others.
- **Speech Recognition:** Matches spoken words to known patterns to convert them into text.

Advantages of KNN

- **Simple to use:** Easy to understand and implement.
- **No training step:** No need to train as it just stores the data and uses it during prediction.
- **Few parameters:** Only needs to set the number of neighbors (k) and a distance method.
- **Versatile:** Works for both classification and regression problems.

Disadvantages of KNN

- **Slow with large data:** Needs to compare every point during prediction.
- **Struggles with many features:** Accuracy drops when data has too many features.
- **Can Overfit:** It can overfit especially when the data is high-dimensional or not clean.

Program:

```
import numpy as np
from collections import Counter
def euclidean_distance(point1, point2):
    return np.sqrt(np.sum((np.array(point1) - np.array(point2))**2))
def knn_predict(training_data, training_labels, test_point, k):
    distances = []
    for i in range(len(training_data)):
        dist = euclidean_distance(test_point, training_data[i])
        distances.append((dist, training_labels[i]))
    distances.sort(key=lambda x: x[0])
    k_nearest_labels = [label for _, label in distances[:k]]
    return Counter(k_nearest_labels).most_common(1)[0][0]
training_data = [[1, 2], [2, 3], [3, 4], [6, 7], [7, 8]]
training_labels = ['A', 'A', 'A', 'B', 'B']
test_point = [4, 5]
k = 3
prediction = knn_predict(training_data, training_labels, test_point, k)
print(prediction)
```

Output:

A

Conclusion: Building a K-Nearest Neighbors (KNN) Classification Model helped me understand how classification works based on similarity between data points. I learned that KNN predicts the class of a new data point by checking the majority class among its K nearest neighbors.